



DuocUC[®] INFORMÁTICA Y
TELECOMUNICACIONES



- **Control de Versiones
del Software**
-

- **El Desastre de la Versión Perdida**

Un desarrollador realiza una actualización crítica en el módulo de gestión de recursos de un hospital, mientras otro, sin saberlo, trabaja sobre una versión antigua del mismo módulo. Al intentar integrar los cambios, se pierden actualizaciones importantes, resultando en errores y conflictos en el código. **Este caos provoca un retraso significativo en la entrega del proyecto, lo que lleva a que el hospital no pueda gestionar adecuadamente sus recursos, afectando la atención a los pacientes y generando una pérdida de confianza y posibles repercusiones legales.**

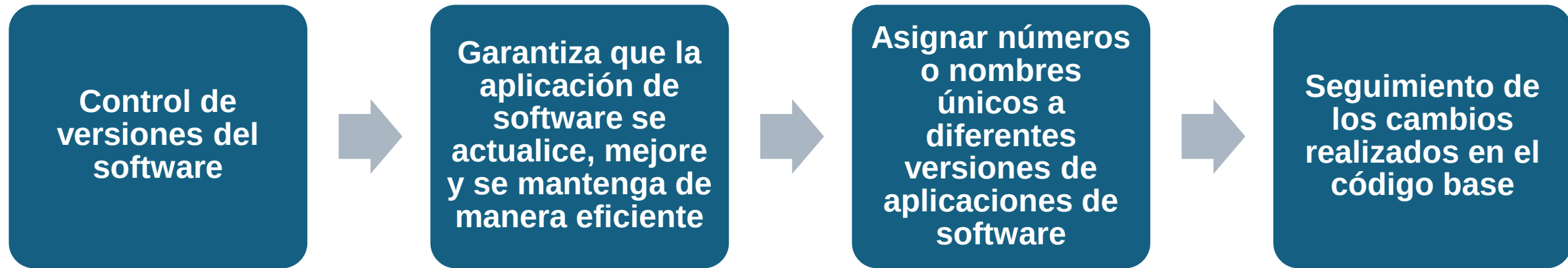
¿Qué medidas preventivas podrían haberse tomado para evitar esta situación?

A black and white photograph of a man in a suit standing in a modern office, holding a tablet and smiling. The office has glass partitions and modern furniture. A blue square is visible in the top right corner of the image.

01

- **Introducción
al Control de
Versiones**

- **¿Qué es el control de versiones de software?**



- **Tipos de Control de Versiones**

Existen varios tipos de control de versiones de software y cada uno tiene sus ventajas y desventajas. Algunos de los tipos más comunes de control de versiones de software se verán a continuación

• Control de Versiones

Secuencial

- Más sencillo
- Cada nueva versión se le asigna un número secuencial, como **1.0, 1.1, 1.2**, etc.
- Fácil de entender e implementar
- No proporciona mucha información sobre los cambios realizados en el software.

Por fecha

- Se basa en la fecha de lanzamiento, como 2022.01.01.
- Ayuda a los desarrolladores y usuarios a realizar un seguimiento de los cambios realizados en el software a lo largo del tiempo
- No proporciona mucha información sobre los cambios realizados en el software.

Semántico

- Se basa en un esquema numérico de tres partes, como 1.0.0
- El primer número representa una versión principal, el segundo número representa una versión menor y el tercer número representa una versión de parche.
- Proporciona más información sobre los cambios realizados en el software
- Ayuda a los desarrolladores y usuarios a comprender el impacto de los cambios en el software.

• Control de Versiones Semántico

01

• **Número de Versión Principal** : se incrementa cuando hay cambios o actualizaciones importantes en el software que no son compatibles con versiones anteriores. Esto significa que cualquier software que dependa de la versión anterior no funcionará con la nueva versión. Por ejemplo, si se descubre una falla de seguridad importante en el software, es posible que sea necesaria una actualización importante de la versión para solucionarla

02

• **Número de versión secundaria**: el número de versión secundaria se incrementa cuando se agregan nuevas características o funcionalidades al software, pero aún son compatibles con versiones anteriores. Esto significa que cualquier software que dependa de la versión anterior seguirá funcionando con la nueva versión. Por ejemplo, si se agrega una nueva función a un software, es posible que sea necesaria una actualización de versión menor para incluirla.

03

• **Número de versión del parche**: el número de versión del parche se incrementa cuando se corrigen errores o se realizan pequeñas actualizaciones en el software, sin cambios importantes ni nuevas características. Esto significa que cualquier software que dependa de la versión anterior seguirá funcionando con la nueva versión. Por ejemplo, si se descubre un error menor en el software, es posible que sea necesario actualizar la versión del parche para solucionarlo.

04

• **Versiones preliminares**: en algunos casos, el software puede tener versiones preliminares que aún no están listas para su uso en producción. Estas versiones se indican agregando un guión y un identificador de versión preliminar, como alfa, beta o rc, al número de versión. Por ejemplo, la versión 1.0.0-alpha.1 indicaría la primera versión alfa de la versión 1.0.0.

05

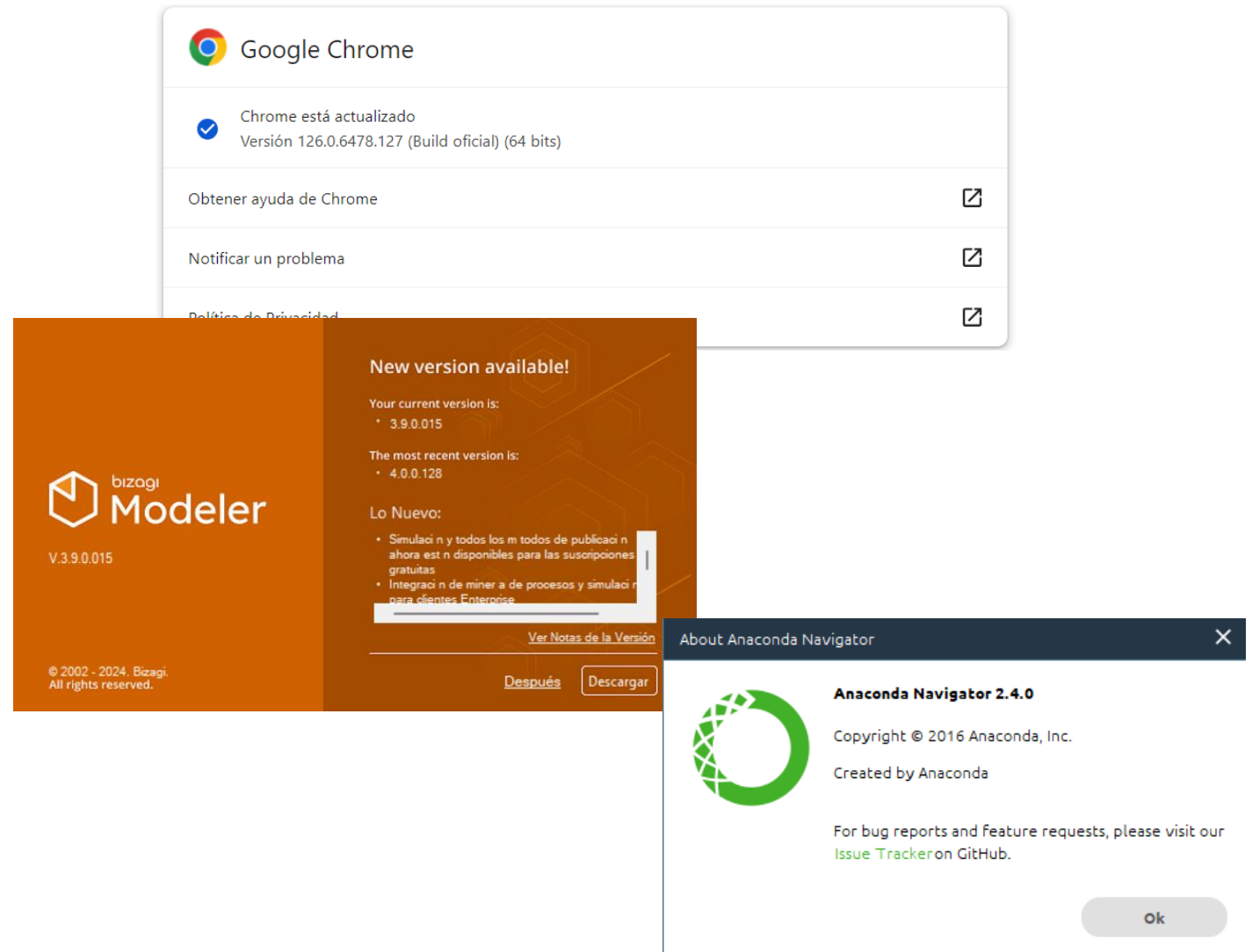
• **Metadatos de compilación**: además de los números de versión y los identificadores previos al lanzamiento, el control de versiones semántico también permite incluir metadatos de compilación en el número de versión. Estos metadatos se indican agregando un signo más y un identificador de compilación, como un hash de confirmación o un número de compilación, al número de versión. Por ejemplo, la versión 1.0.0+20191010 indicaría una compilación de la versión 1.0.0 del 10 de octubre de 2019.

- **Micro Break**



¡1 Cosa!
¿Qué recuerdas?

Analizar con tu profesor los siguientes ejemplos



- **Importancia del Control de Versiones**

- La **gestión eficiente de versiones** es un aspecto crucial del desarrollo software.
- Garantiza que los productos de software se entreguen a los usuarios finales de manera oportuna y eficiente, al tiempo que minimiza el riesgo de errores y fallas.
- En el acelerado entorno de desarrollo de software actual, una gestión eficiente de las versiones puede marcar la diferencia en el éxito de un producto.

En esta sección, exploraremos la importancia de una gestión de lanzamientos eficiente y por qué debería ser una prioridad para los equipos de desarrollo de software.

• Importancia de Control de Versiones eficiente

Minimiza el riesgo de errores y fallos



Mejora el tiempo de comercialización



Mejora la satisfacción del cliente



Facilita la colaboración y la comunicación



Permite la entrega continua

• Micro Break



**¡1 Cosa!
¿Qué recuerdas?**

¿Qué rescato o creo que es importante recordar sobre la gestión eficiente de versiones ?

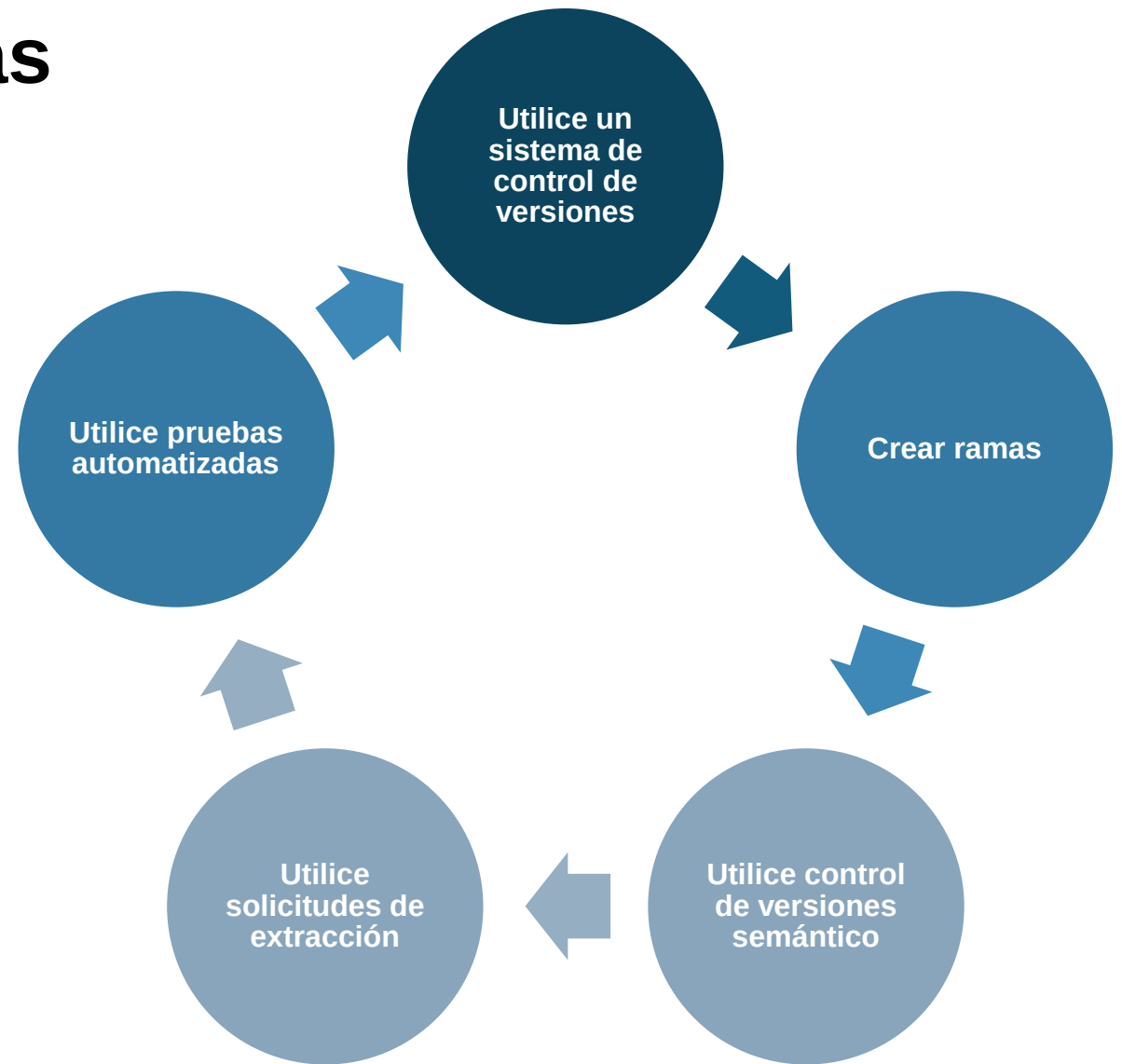


02

- **Prácticas Recomendadas para el Control de Versiones**

• Prácticas Recomendadas

Cuando se trata de desarrollo de software, el control de versiones es un aspecto crítico que garantiza una gestión eficiente de las versiones. Permite a los desarrolladores realizar un seguimiento de los cambios realizados en el código, colaborar con los miembros del equipo y mantener un historial del software. Sin embargo, el control de versiones puede ser complejo y requiere prácticas adecuadas para garantizar su eficacia.



• Utilice un sistema de control de versiones

La primera y más importante práctica es utilizar un sistema de control de versiones (VCS). VCS es una herramienta de software que rastrea los cambios realizados en el código y los almacena en un repositorio central. Proporciona un historial de cambios realizados en el código, permite a los desarrolladores colaborar en la misma base de código y ayuda a administrar diferentes versiones del software.

Hay varios VCS disponibles, **incluidos Git, SVN y Mercurial**. Entre ellos, **Git es el VCS más popular** y utilizado debido a su flexibilidad y facilidad de uso.

• Crear ramas

- Las ramas son una característica poderosa del control de versiones que permite a los desarrolladores trabajar en diferentes versiones del código simultáneamente.
- Permite a los desarrolladores aislar cambios, colaborar en nuevas funciones y corregir errores sin afectar el código base principal. Al crear sucursales, es esencial seguir una convención de nomenclatura que refleje el propósito de la sucursal.
- Por ejemplo, una rama llamada "función/nueva página de inicio de sesión" indica que es una rama para desarrollar una nueva característica de la página de inicio de sesión.

• Utilizar solicitudes de extracción

- Las solicitudes de extracción son una práctica popular en Git que permite a los desarrolladores revisar y discutir los cambios de código antes de fusionarlos en la base de código principal.
- Ayuda a detectar errores y garantizar la calidad del código al permitir que otros miembros del equipo revisen y proporcionen comentarios sobre los cambios.
- Las solicitudes de extracción también permiten a los desarrolladores discutir los cambios y colaborar para mejorar el código

• Utilizar control de versiones semántico

- Como se vio anteriormente , es una práctica que consiste en asignar números de versión a las versiones de software en función de los cambios realizados en el código.
- Sigue un formato de número de versión de tres partes: MAJOR.MINOR.PATCH. El número de versión principal indica cambios significativos que no son compatibles con versiones anteriores, el número de versión secundaria indica nuevas funciones que son compatibles con versiones anteriores y el número de versión del parche indica correcciones de errores y mejoras menores.
- El control de versiones semántico ayuda a comunicar los cambios realizados en el software y garantiza que los usuarios puedan comprender fácilmente el impacto de los cambios

• Utilizar pruebas automatizadas

- Las pruebas automatizadas son una práctica que consiste en utilizar herramientas de software para probar el código automáticamente.
- Ayuda a detectar errores y garantizar la calidad del código ejecutando pruebas repetidamente y brindando comentarios sobre los cambios del código.
- Las pruebas automatizadas también permiten a los desarrolladores detectar regresiones y garantizar que el código base sea estable antes de entregarlo a los usuarios.

• Micro Break



**¡1 Cosa!
¿Qué recuerdas?**

¿Qué rescato o creo que es importante recordar sobre las prácticas de control de versiones ?



03

- **Git y GitHub Básico**

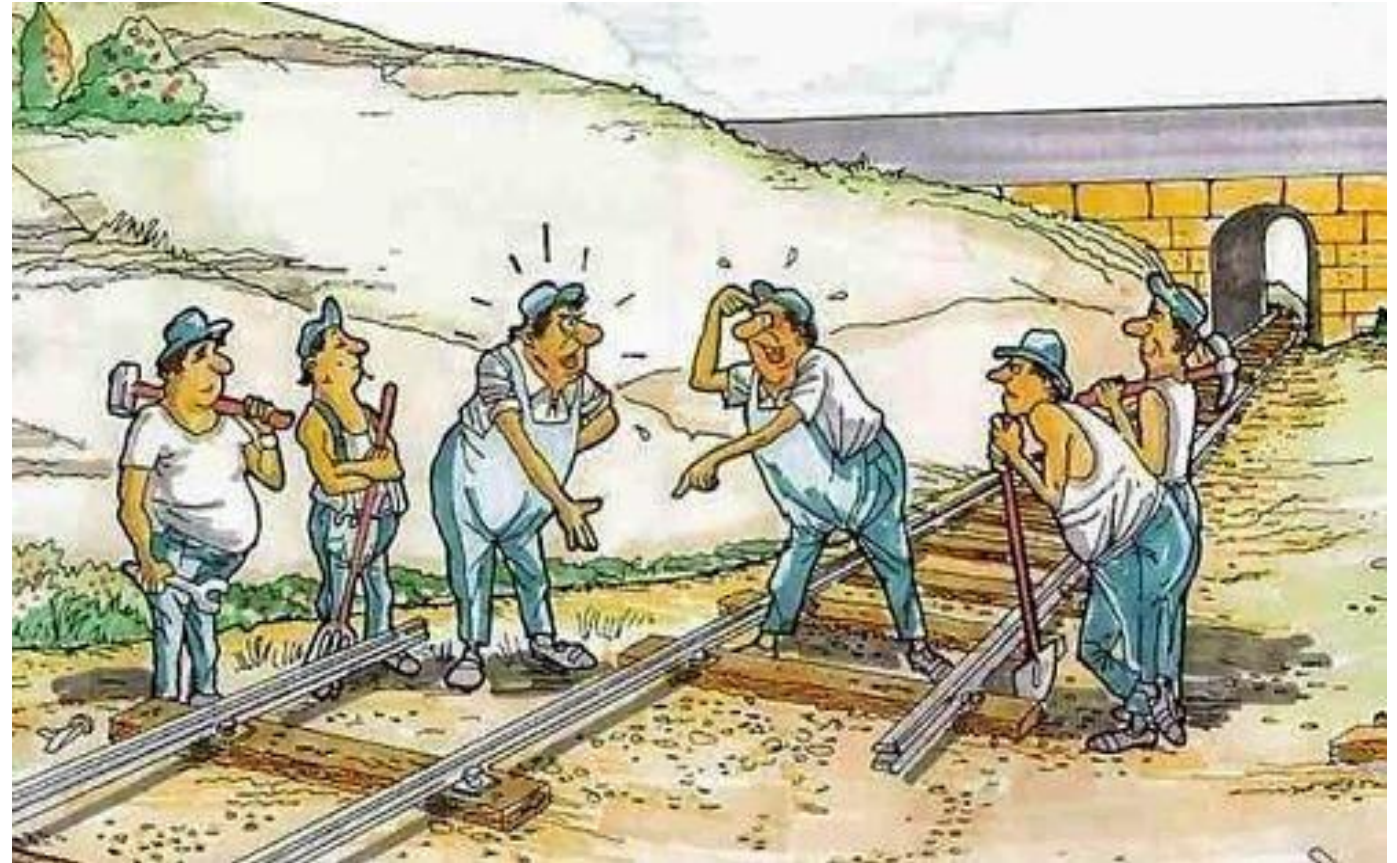
• Git y GitHub

Git es un **sistema de control de versiones (VCS)** distribuido gratuito y de código abierto diseñado para manejar desde proyectos pequeños a muy grandes, con velocidad y eficiencia.

- Nos permite conocer los cambios realizados en los programas o software que vamos desarrollando con el paso del tiempo. Administra las distintas versiones
- Nace como necesidad de control para el Kernel de Linux.
- Coordinar el trabajo entre los diferentes desarrolladores del equipo.
- Repositorios Locales y Remotos.

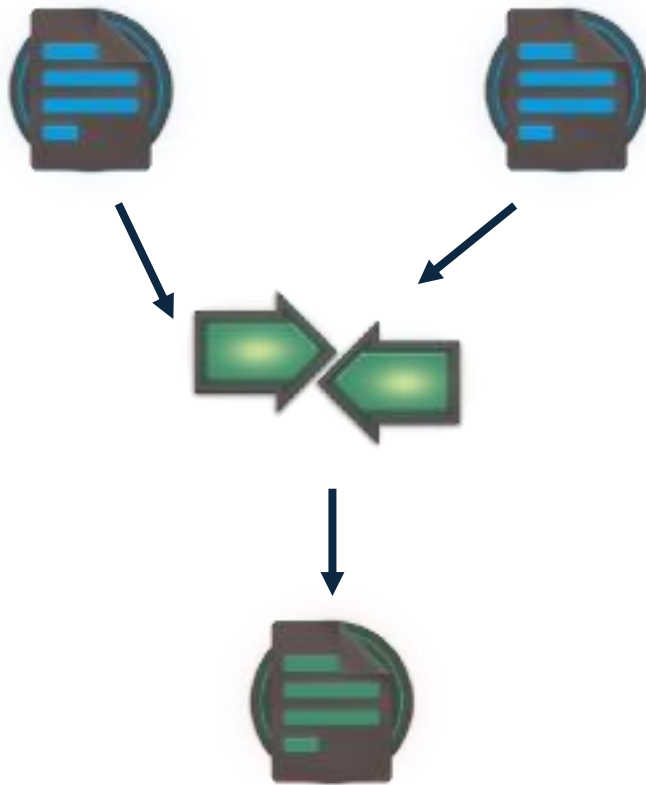
- Resuelve el clásico problema

¡Problemas de colaboración y trabajo en equipo manejando código fuente!

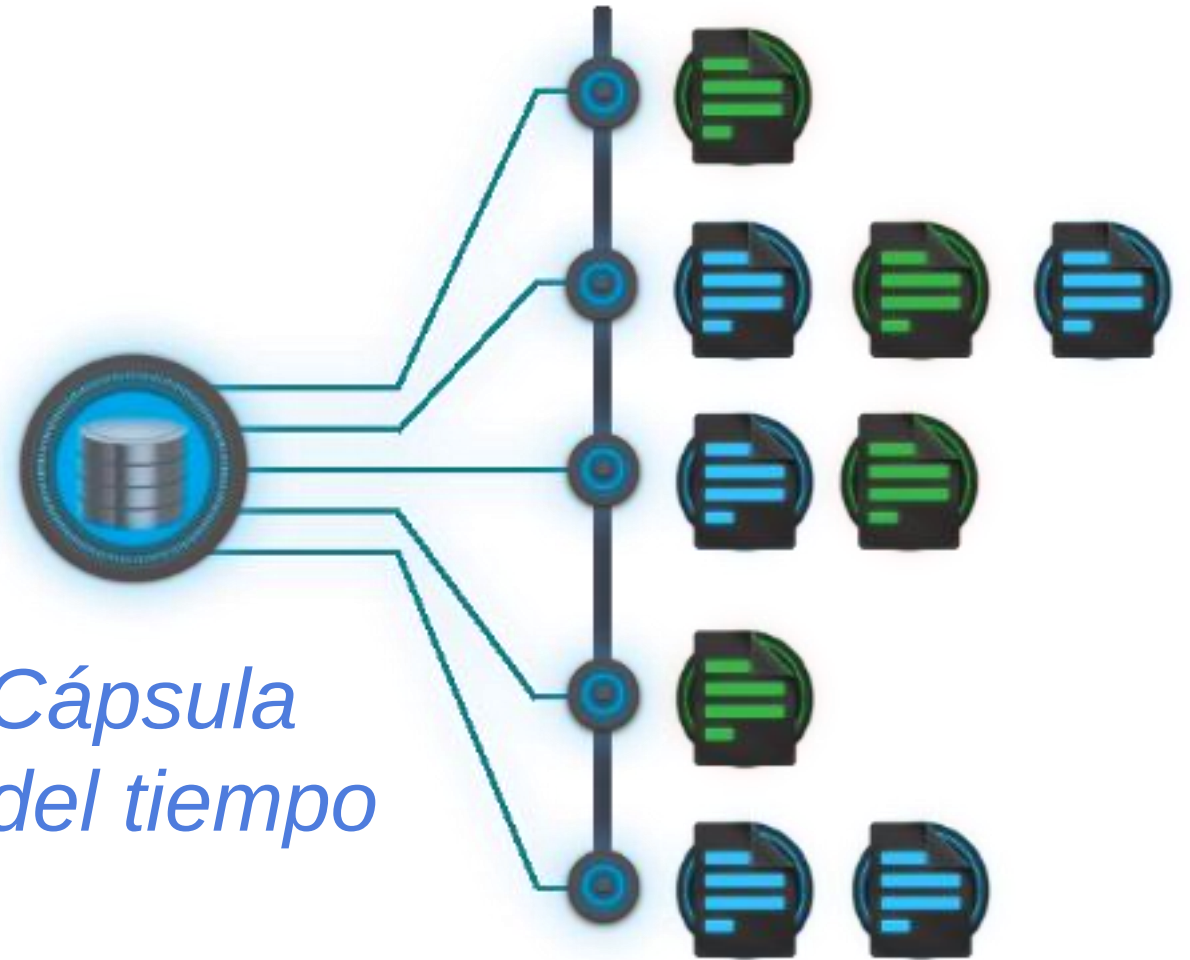


• Sistemas de control de Versiones

Merging

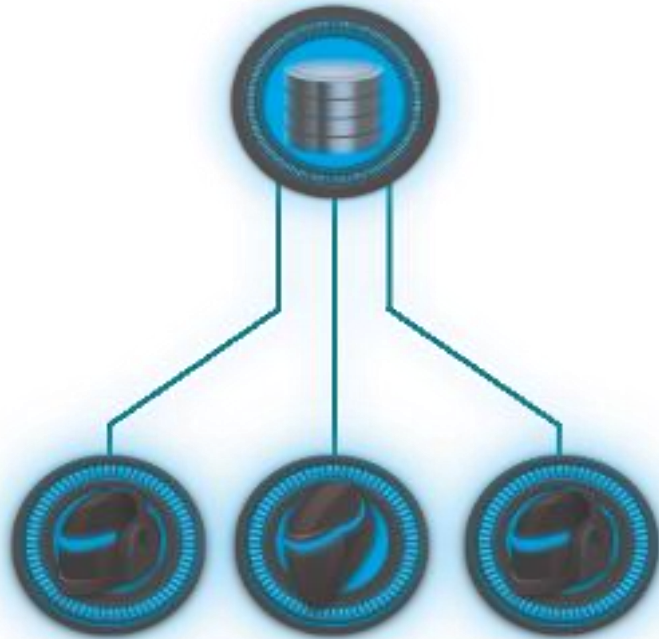


Cápsula del tiempo

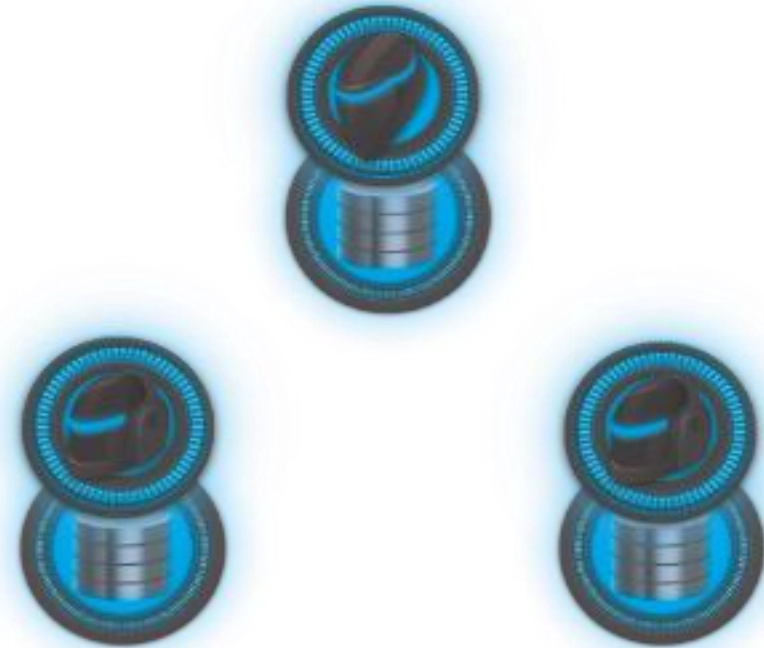


• Ubicación de Repositorios

Repo centralizado

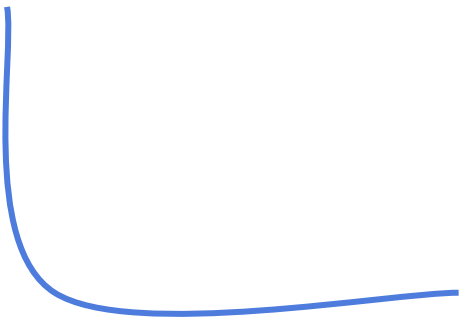


Repos distribuidos



- ¿Qué es Git entonces...?

Git es un Sistema de
control de Versiones
DISTRIBUIDO



en inglés DVCS o
Distributed Version
Control System

• Ventajas

- Sistema distribuido, que permite el trabajo incluso sin conexión.
- Super rápido y ligero, optimizado para hacer operaciones de control muy rápidas.
- Crear ramas y mezclarlas poco propenso a problemas.
- La integridad de la información está asegurada gracias a su modelo de almacenamiento, que permite predecir este tipo de problemas.
- Permite flujos de trabajo muy flexibles.
- El concepto de área de preparación o *staging* permite versionar los cambios como nos convenga, no todo o nada.
- Operaciones locales
- Copias instantáneas

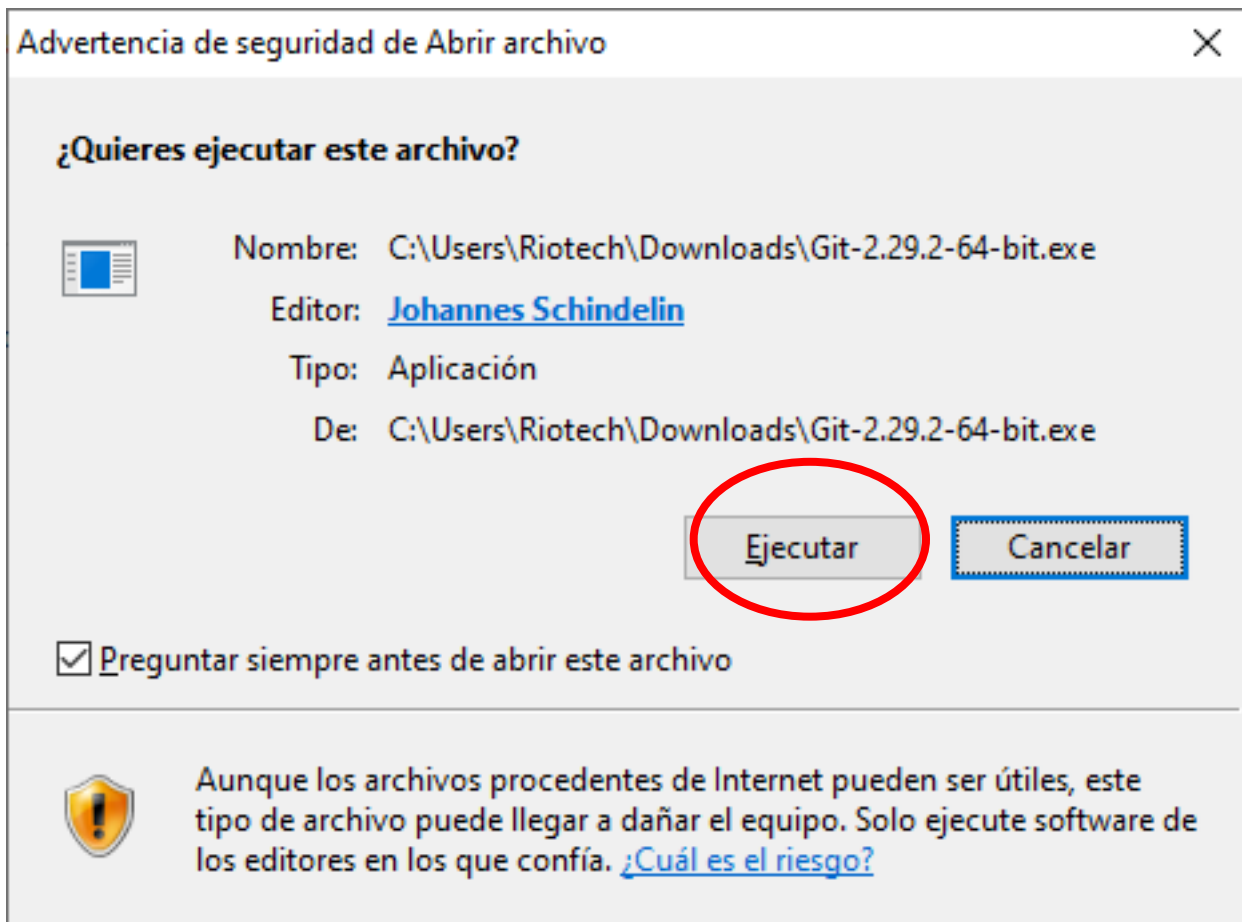
•Instalación de GIT

- Es multiplataforma. Se puede instalar en cualquier sistema operativo: Linux, OS X (Mac) y Windows.
- Automáticamente se descargará la versión más adecuada.

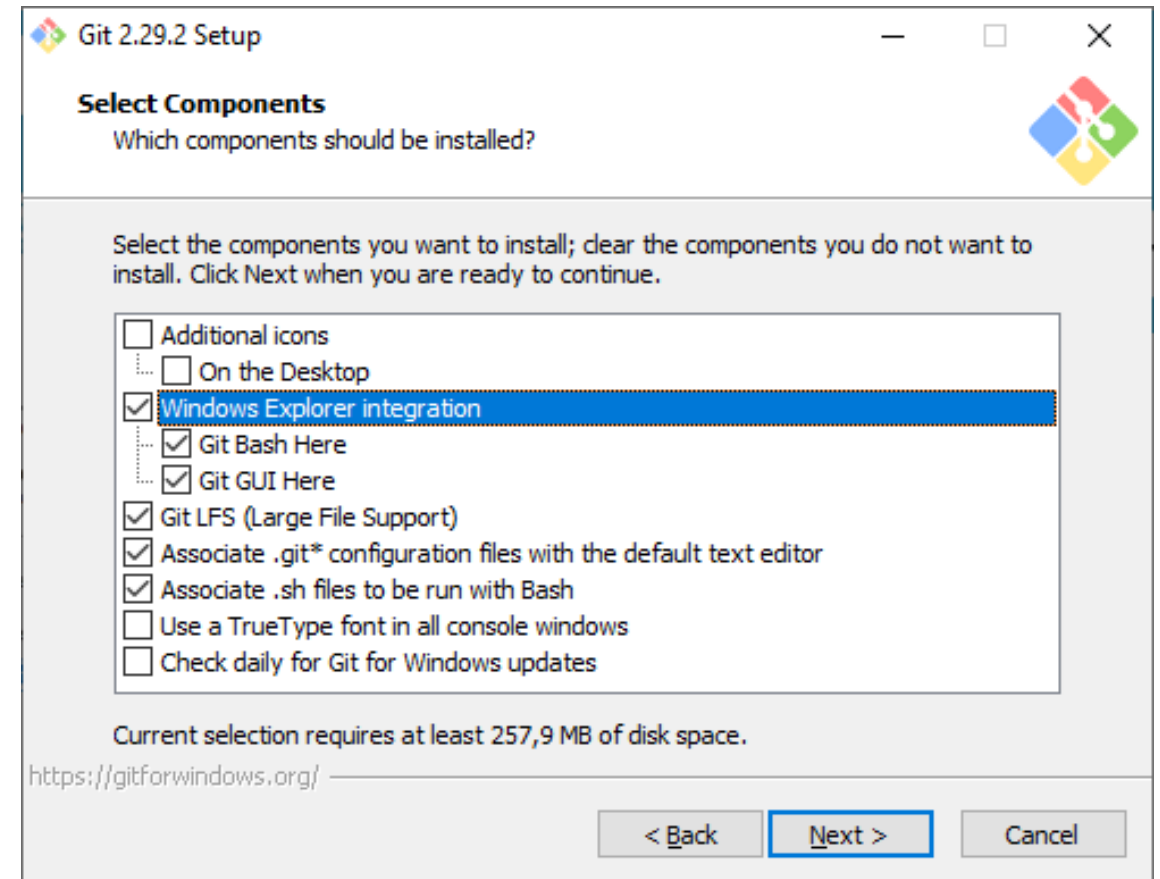
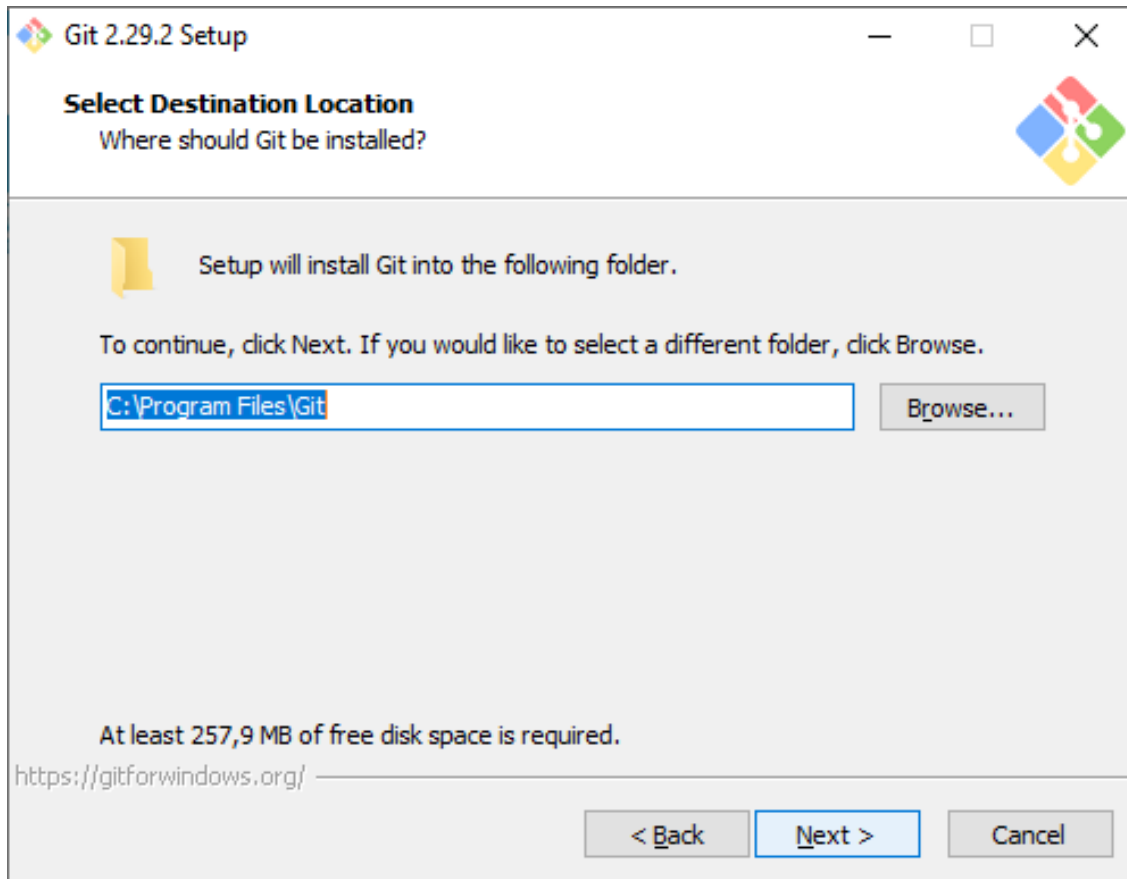


<https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Instalaci%C3%B3n-de-Git>

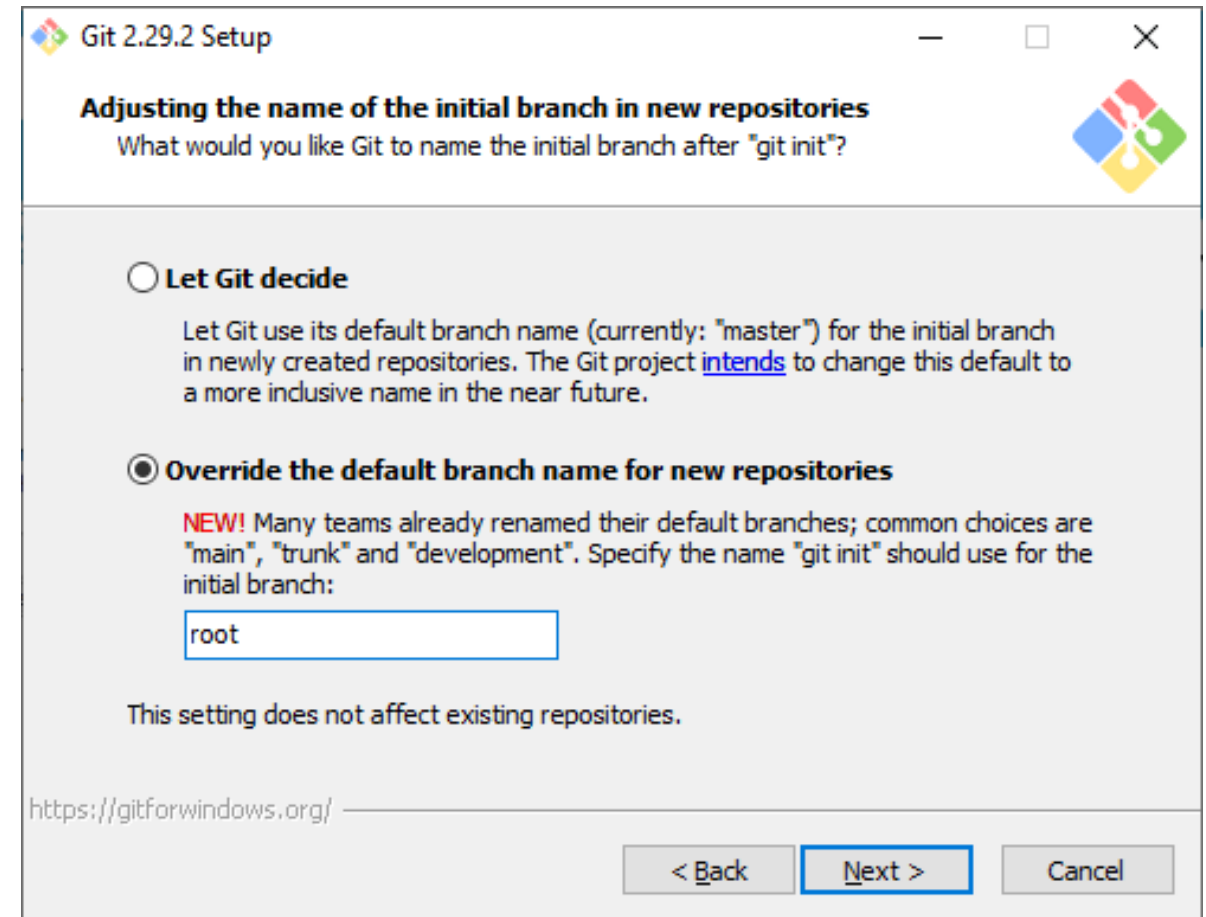
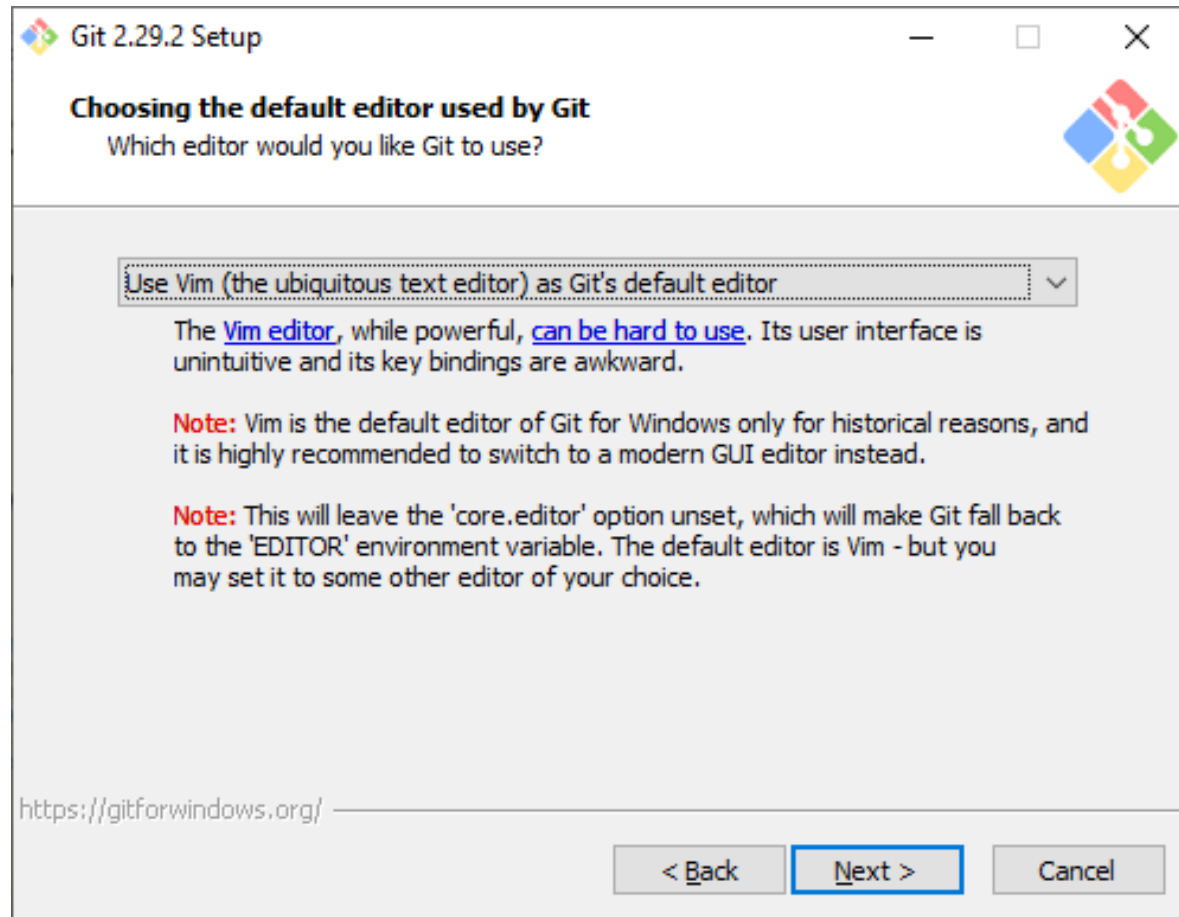
•Instalación de GIT



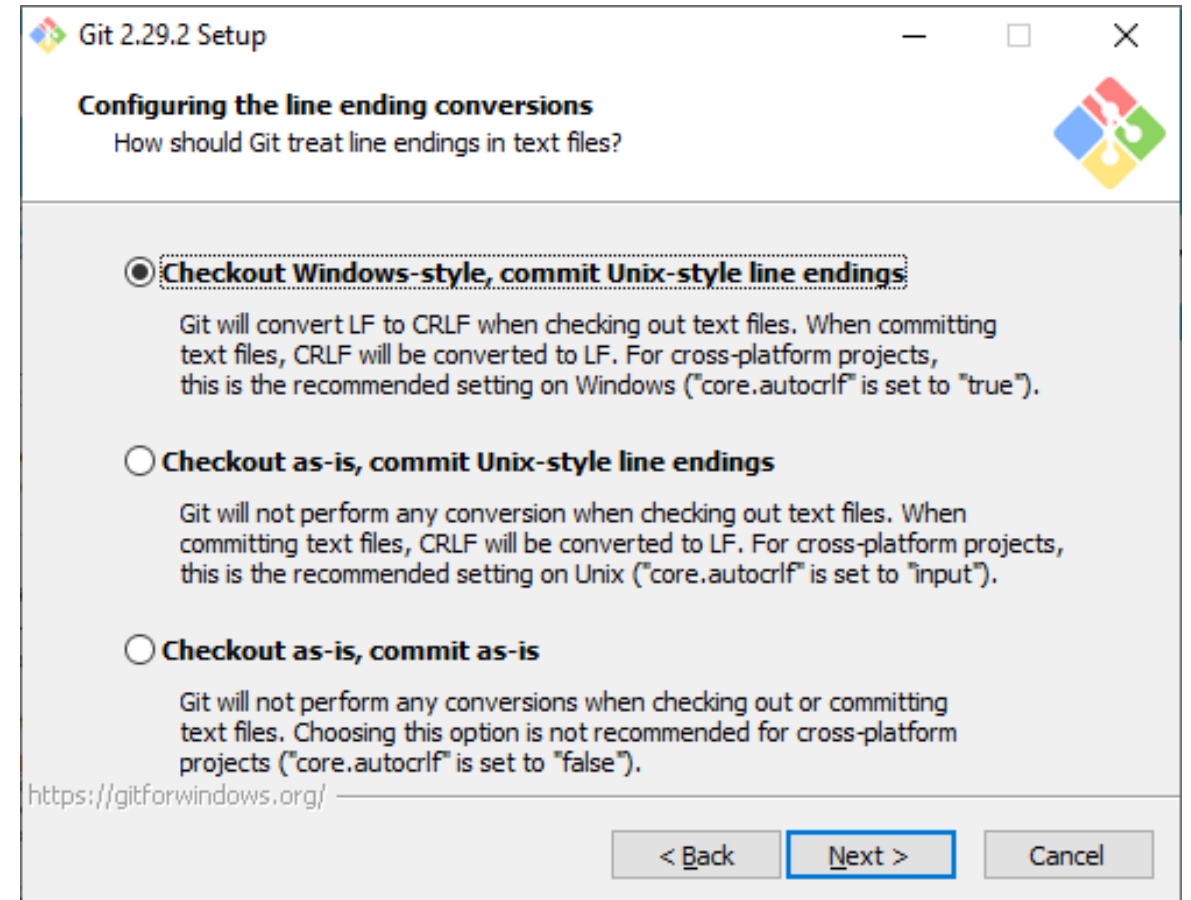
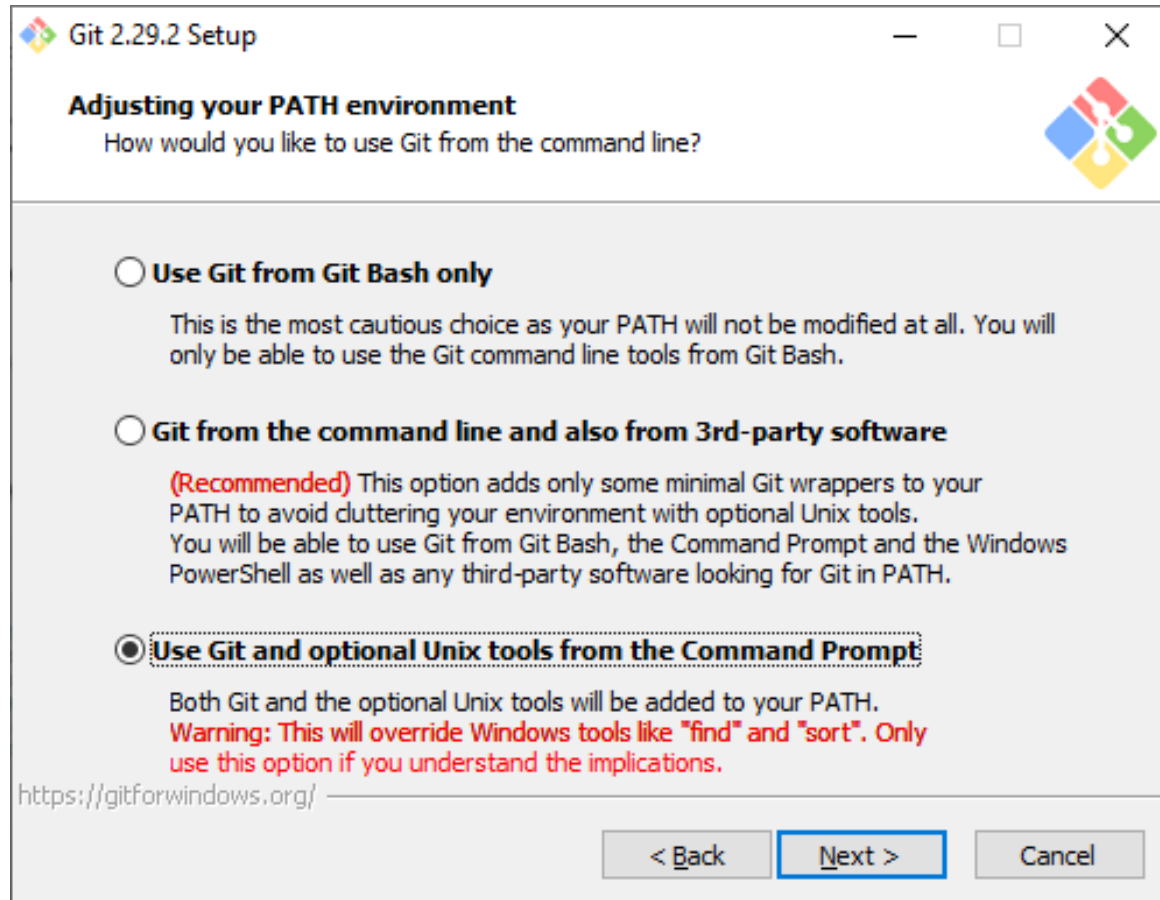
•Instalación de GIT



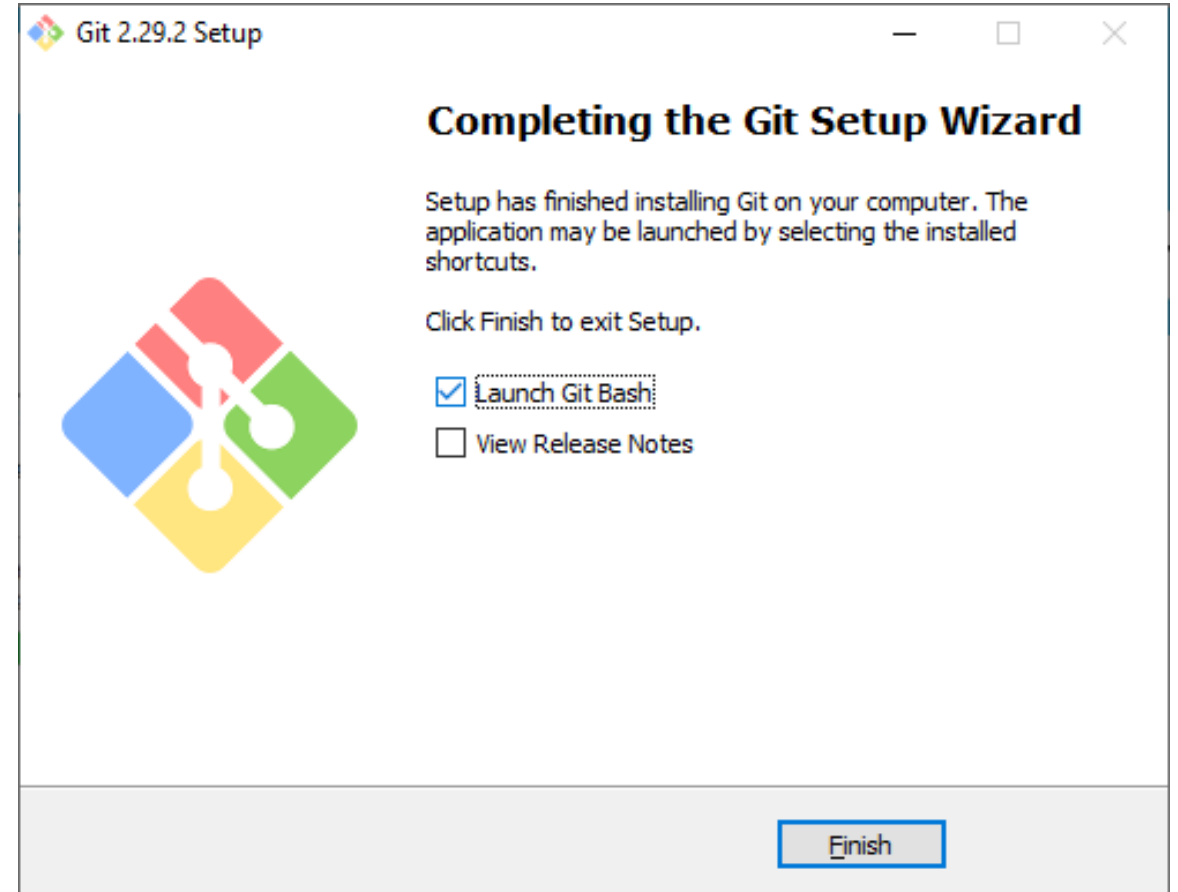
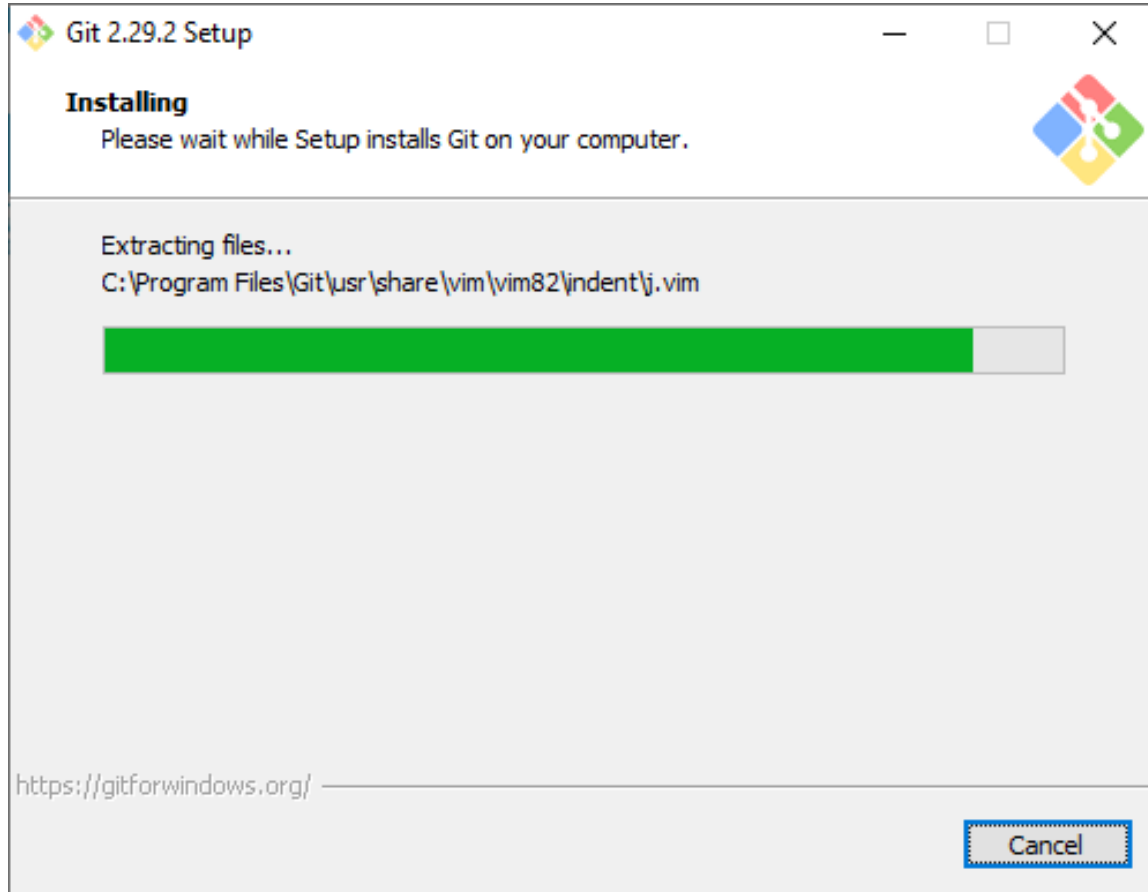
•Instalación de GIT



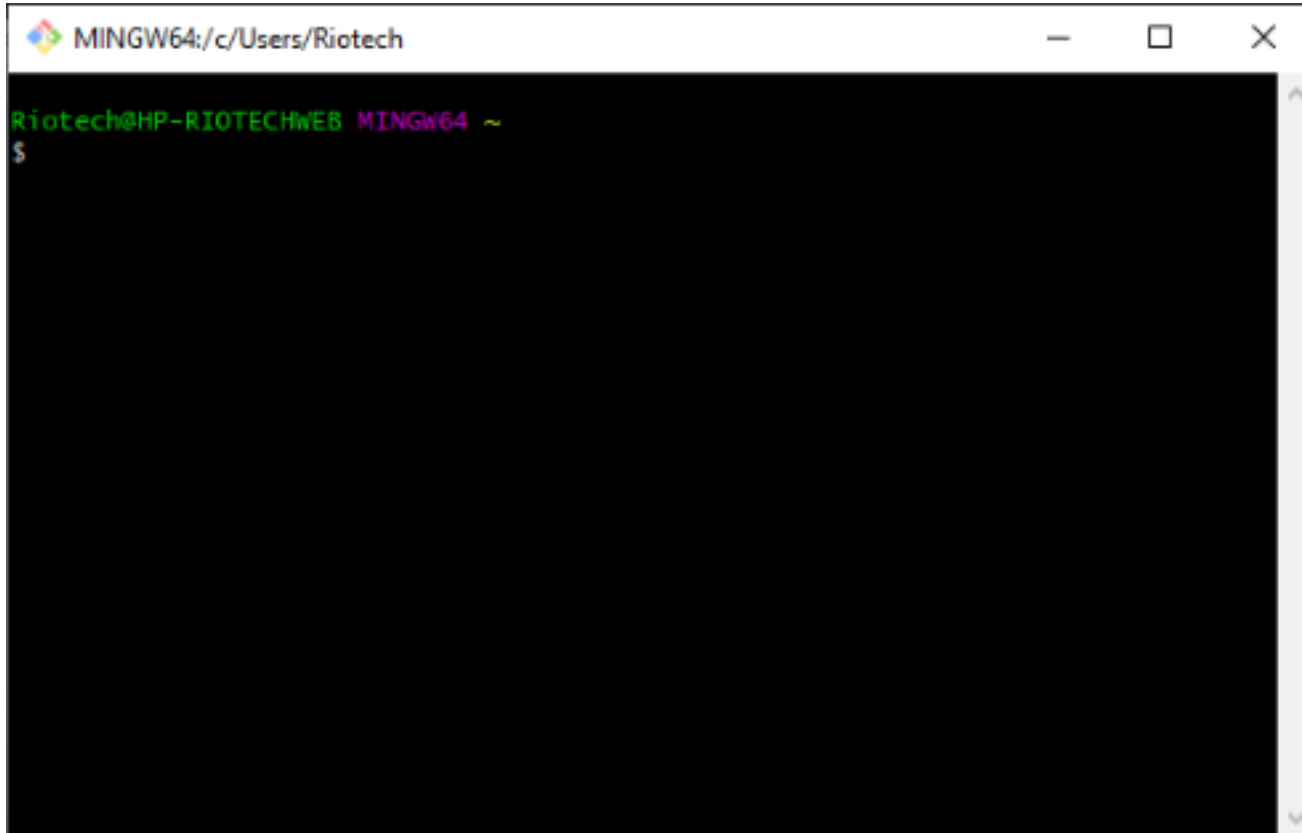
•Instalación de GIT



•Instalación de GIT



•Instalación de GIT

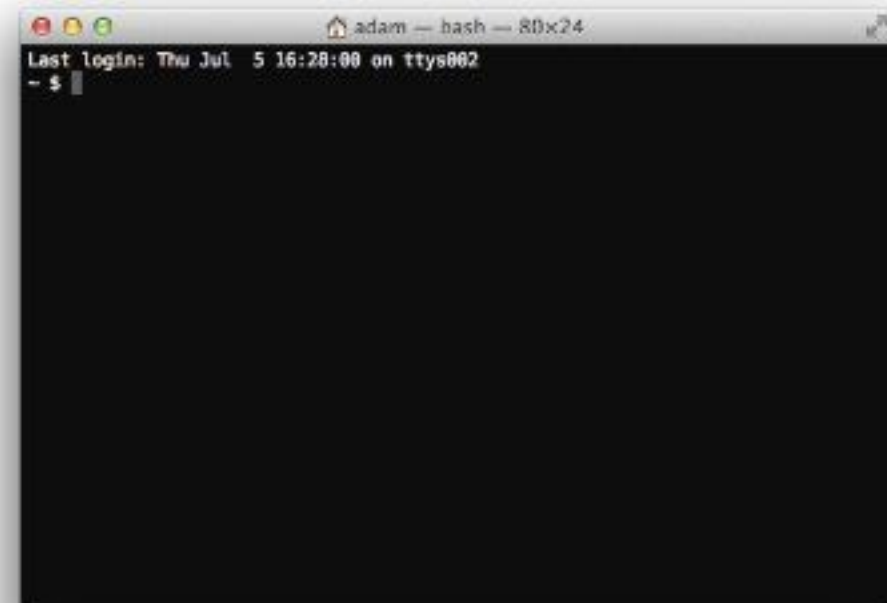


A screenshot of a MINGW64 terminal window. The title bar at the top reads "MINGW64:/c/Users/Riotech". The terminal content shows the prompt "Riotech@HP-RIOTECHWEB MINGW64 ~" followed by a dollar sign "\$" on the next line, indicating it is ready for a command.

ls: lista los directorios y archivos
pwd: directorio actual

•Trabajando con Git

- Desde la línea de comandos
- Existen herramientas con interfaz de usuario gráfica
- Está integrada a la mayoría de los IDE



• Ayuda desde la línea de comandos

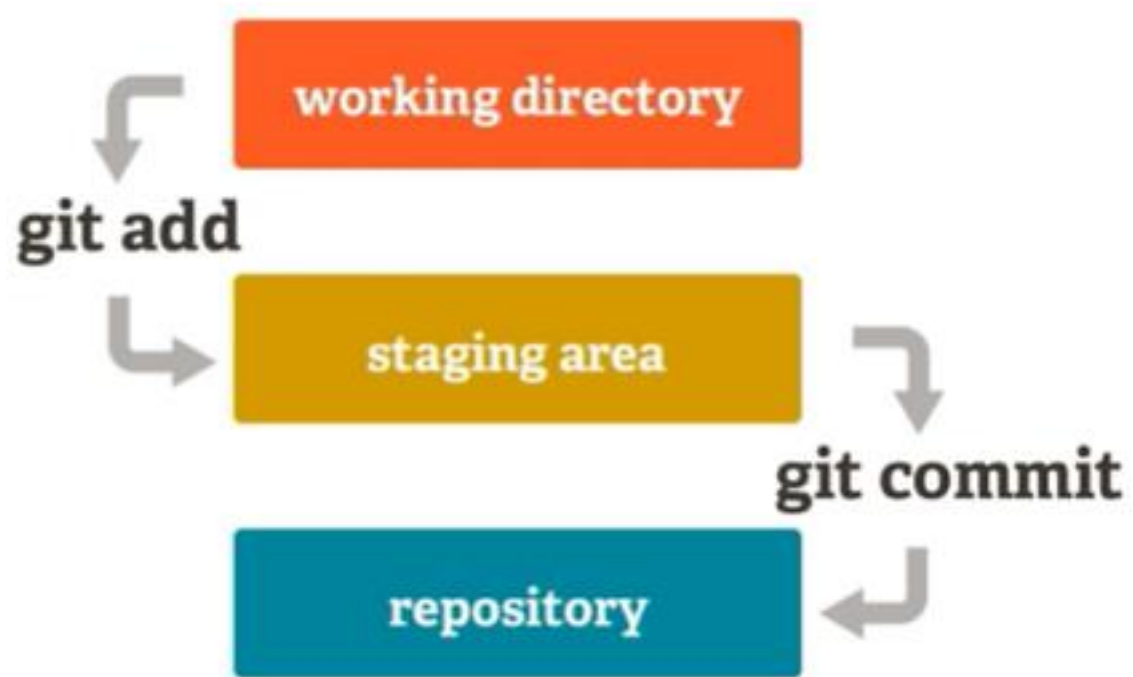
```
λ git help
```

```
usage: git [--version] [--help] [-C <path>] [-c <name>=<value>]
        [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
        [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--bare]
        [--git-dir=<path>] [--work-tree=<path>] [--namespace=<name>]
        <command> [<args>]
```

These are common Git commands used in various situations:

...

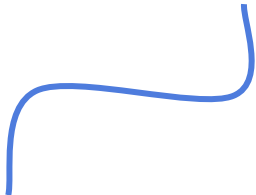
• Comandos Básicos



- `git init`
- `git add <file>`
- `git status`
- `git commit`
- `git push`
- `git pull`
- `git clone`

- **git help**

λ git help config



se le puede
pasar cualquier
comando de git

git-config(1) Manual Page

NAME

git-config - Get and set repository or global options

SYNOPSIS

```
git config [<file-option>] [--type=<type>] [--show-origin] [--show-scope] [-z | --null] name value
git config [<file-option>] [--type=<type>] --add name value
git config [<file-option>] [--type=<type>] --replace-all name value [value_regex]
git config [<file-option>] [--type=<type>] [--show-origin] [--show-scope] [-z | --null] --get name
git config [<file-option>] [--type=<type>] [--show-origin] [--show-scope] [-z | --null] --get-regexp regex
git config [<file-option>] [--type=<type>] [--show-origin] [--show-scope] [-z | --null] [--name-only] --get-regexp regex
```

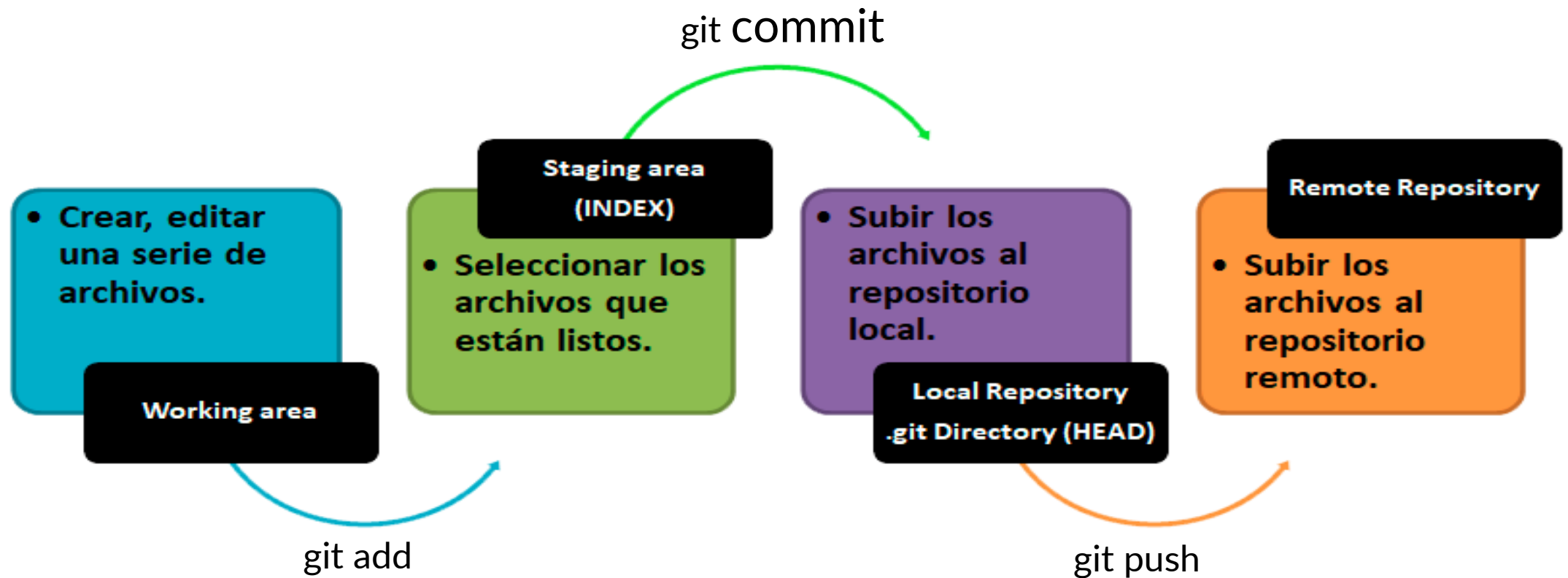
•Configurando Git

```
λ git config --global user.name "Nombre Apellido"
```

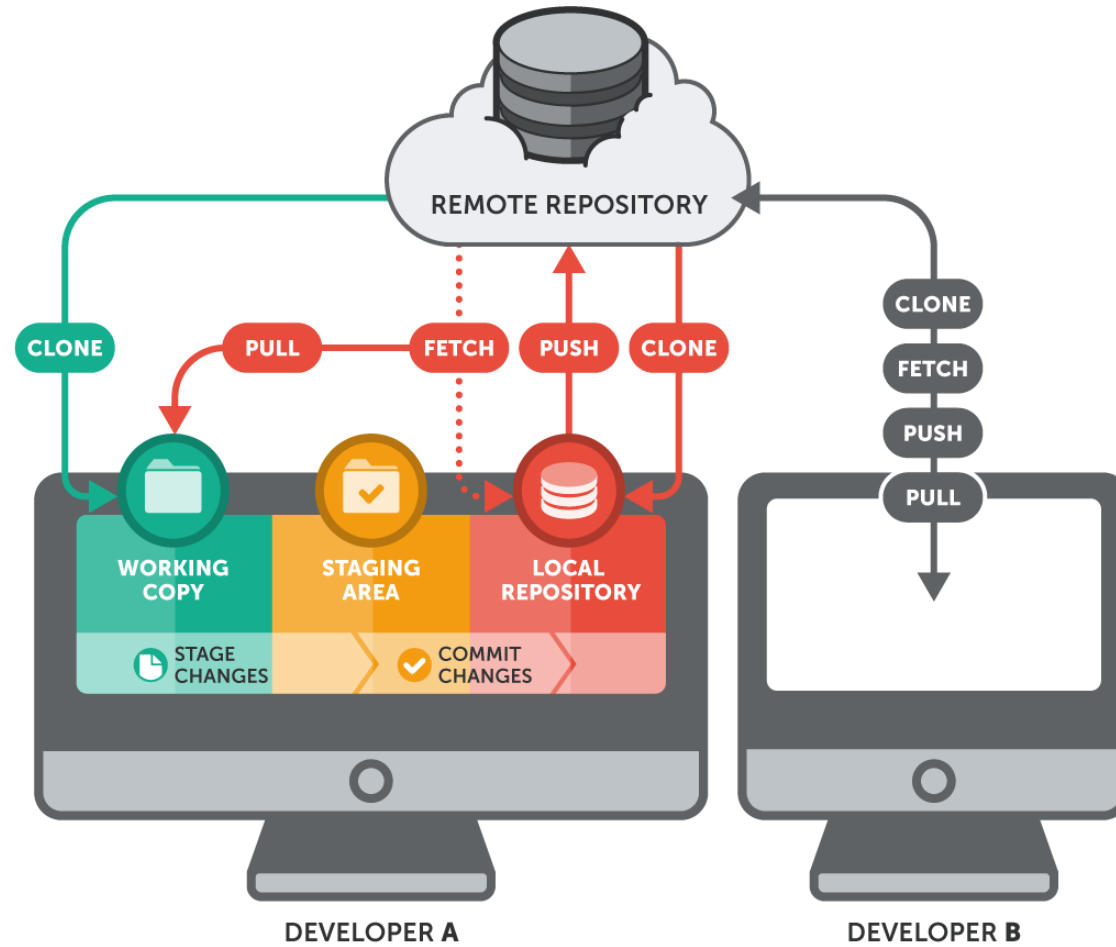
```
λ git config --global user.email "nombre@ejemplo.com"
```

```
λ git config --global color.ui true
```

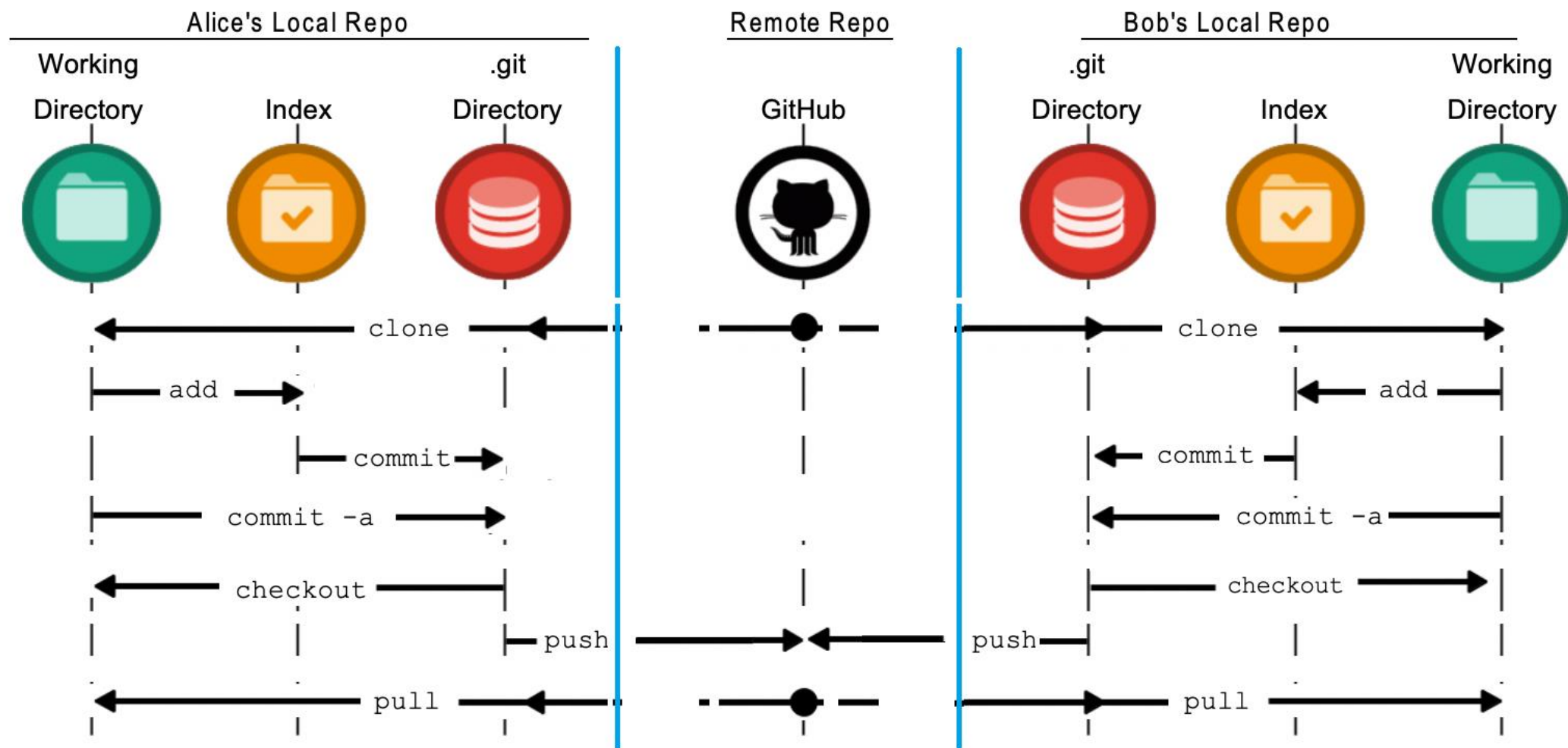
• Flujo de Trabajo



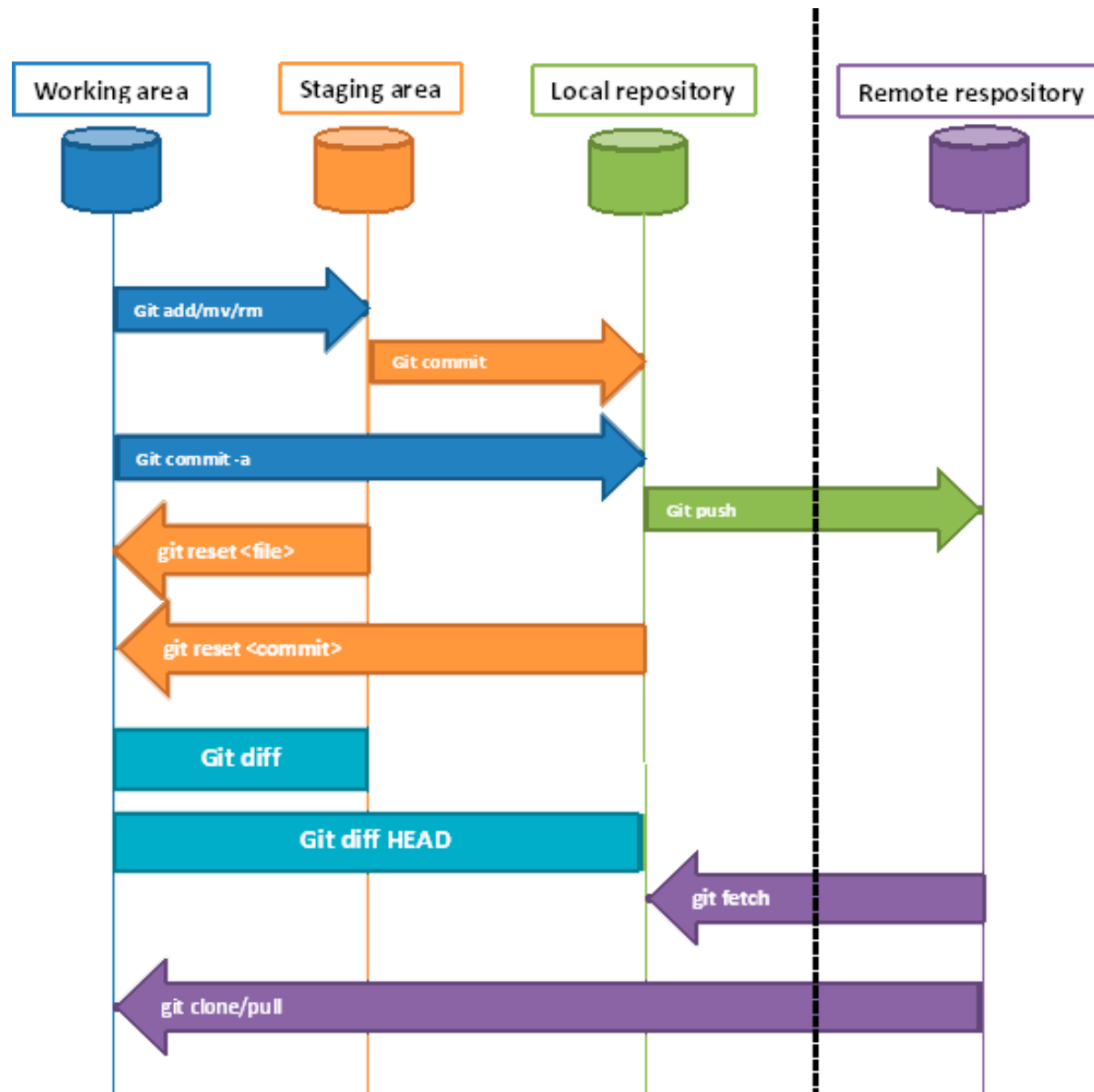
• Flujo de Trabajo



• Flujo de Trabajo



• Flujo de trabajo



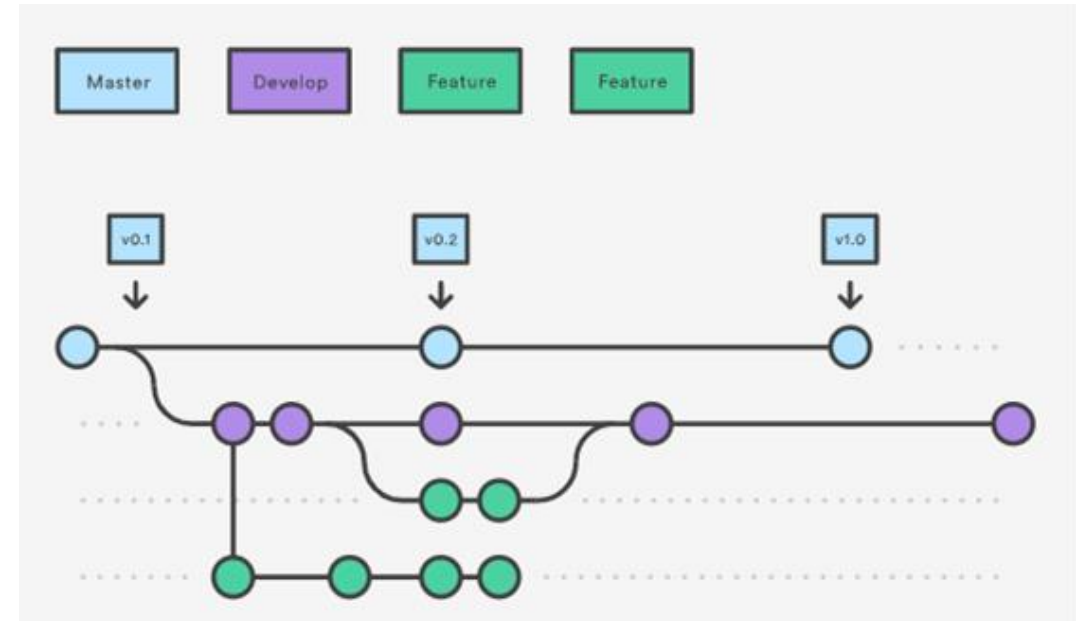
• ¿Cómo colaborar en un proyecto de Git?



<https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>

• Flujo de Trabajo en Git

Como punto de partida, te proponemos analizar el flujo de trabajo diseñado por Vincent Driessen en el cual, proporciona un modelo sólido para gestionar proyectos.



Para profundizar clic en el siguiente enlace: <https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow>

• Git: la guía sencilla

<https://rogerdudler.github.io/git-guide/index.es.html>

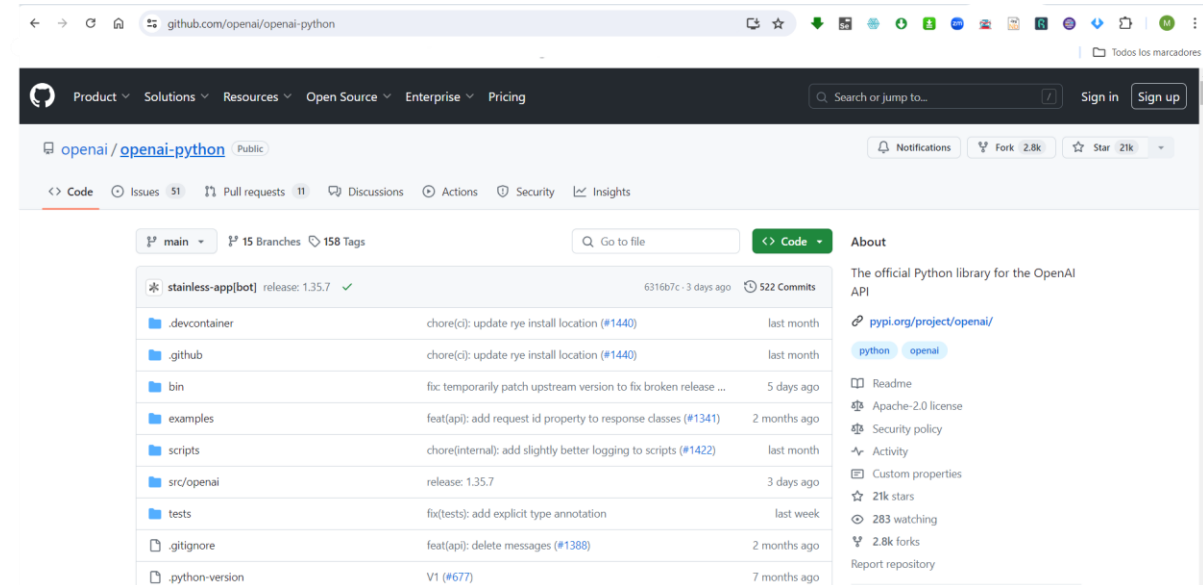


<https://www.atlassian.com/es/git/tutorials/learn-git-with-bitbucket-cloud>

• Github / Git

¿Cómo funcionan Git y GitHub juntos?

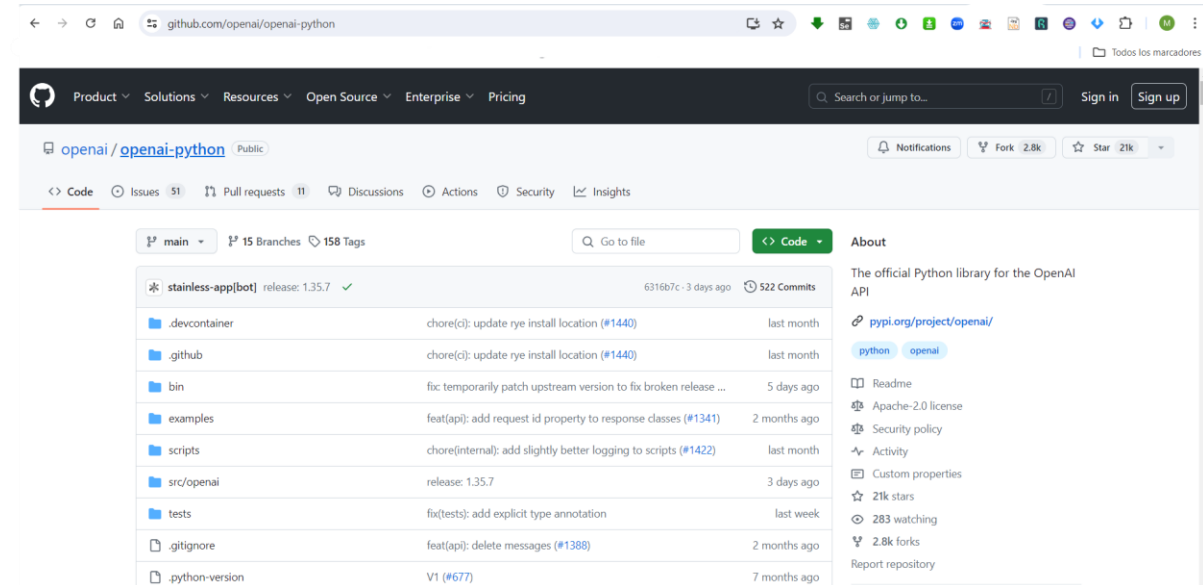
Al cargar archivos en GitHub, los almacenas en un "repositorio de Git". Esto significa que al realizar cambios (o "compromisos") en los archivos de GitHub, Git se iniciará automáticamente para realizar un seguimiento y administrar los cambios.



• Github / Git

Hay muchas acciones relacionadas con Git que puedes completar en GitHub directamente en el navegador, como crear un repositorio de Git, crear ramas y cargar y editar archivos.

Sin embargo, la mayoría de las personas trabajan en sus archivos localmente (en su propio ordenador), luego sincronizan continuamente estos cambios locales y todos los datos de Git relacionados, con el repositorio central "remoto" en GitHub. Hay muchas herramientas que puedes usar para hacerlo, como GitHub Desktop.



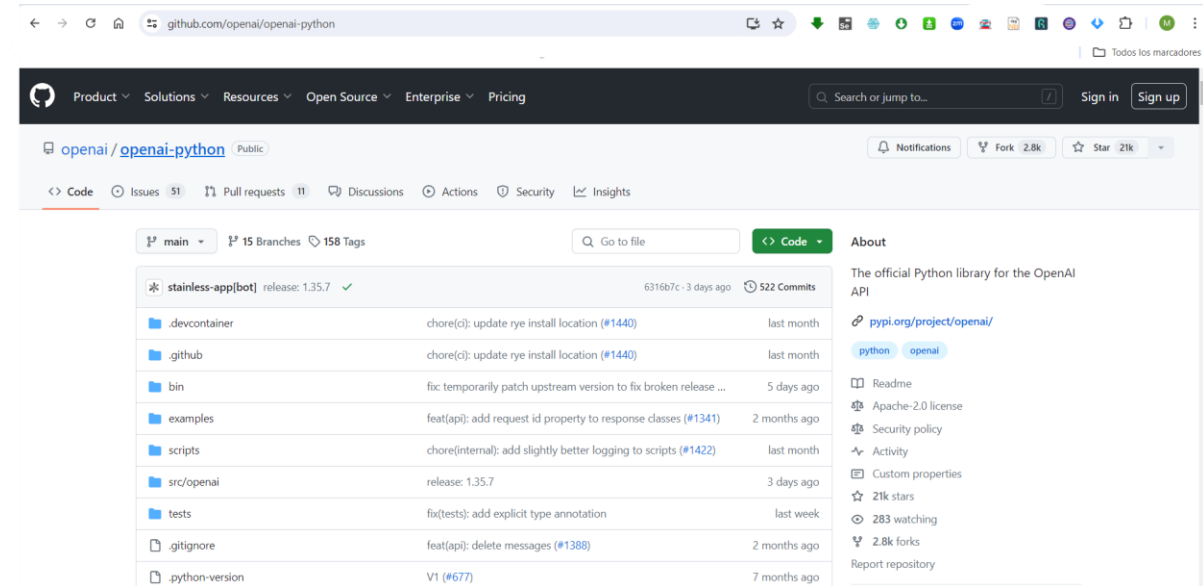
• Github / Git

Cuando empieces a colaborar con otros y todos necesiten trabajar en el mismo repositorio al mismo tiempo, haréis lo siguiente continuamente:

- **Extraer** todos los cambios más recientes realizados por los colaboradores del repositorio remoto en GitHub.

- **Transferir de vuelta** los cambios al mismo repositorio remoto en GitHub.

Git determina cómo combinar inteligentemente este flujo de cambios y GitHub te ayuda a administrar el flujo a través de características como "solicitudes de cambios".



- **Micro Break**



- **Ponte a prueba**

Crea tu propia cuenta GitHub

Acerca de tu cuenta personal de GitHub.com

Para empezar a trabajar con GitHub, deberás crear una cuenta personal gratuita en GitHub.com y comprobar la dirección de correo electrónico.

Cada persona que utilice GitHub.com deberá iniciar sesión en una cuenta personal. Tu cuenta personal es tu identidad en GitHub.com y tiene un nombre de usuario y perfil. Por ejemplo, consulte [el perfil de @octocat](#).

Más adelante, puedes explorar los diferentes tipos de cuentas que GitHub ofrece y decidir si necesitas un plan de facturación. Para obtener más información, vea «[Tipos de cuentas de GitHub](#)» y «[Planes de GitHub](#)».

Ten en cuenta que los pasos de este artículo no se aplican a Enterprise Managed Users. Si tu cuenta de GitHub la ha creado tu empresa para ti, puedes omitir este artículo y continuar con "[Hola mundo](#)".

Registrarse para una cuenta personal nueva

- 1 Vaya a <https://github.com/>.
- 2 Haga clic en Registrarse.
- 3 Sigue las indicaciones para crear tu cuenta personal.

Durante el registro, se te pedirá que verifiques tu dirección de correo electrónico. Sin una dirección de correo electrónico verificada, no podrás completar algunas tareas básicas de GitHub, como crear un repositorio.

04

- Conclusiones y Resumen

• Resumen

1. **Tipos de Control de Versiones** : Se describen tres tipos: secuencial, por fecha y semántico, cada uno con sus ventajas y desventajas. El control semántico proporciona más información sobre los cambios realizados en el software.
2. **Importancia del Control de Versiones**: Una gestión eficiente de versiones minimiza errores, mejora la satisfacción del cliente, facilita la colaboración, permite la entrega continua y acelera el tiempo de comercialización.
3. **Prácticas Recomendadas para el Control de Versiones**: Incluyen el uso de sistemas de control de versiones, creación de ramas, solicitudes de extracción, control de versiones semántico y pruebas automatizadas para garantizar la calidad del código y la colaboración efectiva.
4. **Git y GitHub Básico**: Git es un sistema de control de versiones distribuido que permite rastrear cambios y coordinar el trabajo en equipo. GitHub es una plataforma para almacenar y gestionar repositorios de Git, facilitando la colaboración y sincronización de cambios.

• Bibliografía

Lynn, B. (2010). *Git Magic*. Createspace Independent Publishing Platform.

Acerca de GitHub y Git. (s/f). <https://docs.github.com/es/get-started/start-your-journey/about-github-and-git>

(S/f-e). Fastercapital.com. Recuperado el 30 de junio de 2024, de <https://fastercapital.com/es/contenido/Control-de-versiones-de-software--una-guia-para-una-gestion-eficiente-de-versiones.html>

Git y Github basico.pptx. (s/f). SlideShare. Recuperado el 30 de junio de 2024, de <https://es.slideshare.net/slideshow/git-y-github-basicopptx/257034016>

- **Próxima clase**

- La próxima clase trabajaremos en una actividad práctica basada en control de versiones



DuocUC[®]

CERCANÍA. LIDERAZGO. FUTURO.

duoc.cl

7 AÑOS
ACREDITADO



DESDE AGOSTO 2017 HASTA AGOSTO 2024.
DOCENCIA DE PREGRADO. GESTIÓN
INSTITUCIONAL. VINCULACIÓN CON EL MEDIO.